

Отчёт по лабораторной работе №1

Математические основы защиты информации и информационной безопасности

Ли Хан

Содержание

1	Цель работы	5
2	Ход выполнения	6
2.1	Реализация шифра Цезаря (Шифр Цезаря)	6
2.2	шифра Цезаря компилятора	7
2.3	результат	8
2.4	Реализация шифра Атбаш (Шифр Атбаш)	8
2.5	шифра Атбаш компилятора	9
2.6	шифра Атбаш результат	10
3	Вывод	11

Список иллюстраций

2.1	шифра Цезаря компилятора	7
2.2	шифра Цезаря результат	8
2.3	шифра Атбаш компилятора	9
2.4	шифра Атбаш результат	10

Список таблиц

1 Цель работы

Целью данной работы является изучение основных принципов функционирования шифров простой замены, а также освоение математических моделей шифра Цезаря и шифра Атбаш. В ходе практики необходимо реализовать алгоритмы шифрования и дешифрования для текстов на русском языке и проверить логику применения моноалфавитных подстановок в защите информации.

2 Ход выполнения

2.1 Реализация шифра Цезаря (Шифр Цезаря)

Суть шифра Цезаря заключается в циклическом сдвиге каждого символа открытого текста на фиксированное расстояние k (ключ) в пределах алфавита.

- Математическая модель: Процесс шифрования описывается формулой $E(i) = (i + k) \bmod m$, где i — индекс символа, а m — мощность (длина) алфавита.
- Этапы реализации:
 - Создание индексной таблицы, содержащей 33 буквы русского алфавита.
 - Ввод произвольного целого числа в качестве ключа k .
 - Использование операции модуля для обработки сдвига, что гарантирует циклический переход при выходе индекса за границы алфавита.
 - Сохранение пробелов и специальных знаков в исходном виде без выполнения вычислений.

2.2 шифра Цезаря компилятора

```
[5]
✓ 0
秒
def caesar_cipher(text, key, alphabet):
    """
    此函数实现了凯撒密码的加密与解密逻辑，它是通过将字符在字母表中按固定位移里移动来实现的。
    Эта функция реализует логику шифра Цезаря, который р

    result = ""
    # 首先，我们要获取字母表的总长度，它是我们后续进行循环移位（模运算）的基准。
    # Во-первых, мы получаем общую длину алфавита, которая.
    n = len(alphabet)

    for char in text:
        # 我们需要判断当前字符是否在预设的字母表中，从而决定是否对其进行加密处理。
        # Нам нужно проверить, входит ли текущий символ :
        if char.lower() in alphabet:

            # 第一步，我们定位该字符在字母表中的原始索引位置，这是计算位移的起点。
            # Шаг 1: мы находим исходный индекс этого сим
            idx = alphabet.find(char.lower())

            # 第二步，我们将原始索引与密钥相加，并通过对字母表长度取模，确保位移超出末尾时能回到开
            # Шаг 2: мы складываем индекс с ключом и испо
            new_idx = (idx + key) % n

            # 第三步，我们根据计算出的新索引提取加密字符，并判断原字符的大小写状态以保持格式一致。
            # Шаг 3: мы извлекаем зашифрованный символ по
            new_char = alphabet[new_idx]
            result += new_char.upper() if char.isupper() else new_char
        else:
            # 如果字符是空格或标点符号，我们则不进行任何修改，直接将其原样拼接到结果中。
            # Если символ является пробелом или знаком п
            result += char

    return result
```

Рис. 2.1: шифра Цезаря компилятора

2.3 результат

```
# --- 演示与测试逻辑 / Демонстрация и тестирование ---

# 我们定义一个完整的俄语字母表，包含33个字母，作为程序运行的基础词库。
# Мы определяем полный русский алфавит из 33 букв,
ru_alphabet = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"

# 这里演示凯撒密码：输入明文和密钥，函数将按照我们解释的位移逻辑生成密文。
# Здесь демонстрация шифра Цезаря: вводим текст и
print("Caesar:", caesar_cipher("Пришел увидел победил", 3, ru_alphabet))

# 这里演示阿特巴什密码：只需输入明文，函数将通过镜像映射生成加密后的结果。
# Здесь демонстрация шифра Атбаш: достаточно ввес
print("Atbash:", atbash_cipher("Саратов", ru_alphabet))

... Caesar: Тулызо цел жзо тс дз жло
    Atbash: Няо ямр э
```

Рис. 2.2: шифр Цезаря результат

2.4 Реализация шифра Атбаш (Шифр Атбаш)

Шифр Атбаш является специфическим видом моноалфавитной подстановки, который не требует числового ключа и основывается на зеркальном отражении алфавита.

- Описание принципа: Символы алфавита заменяются по принципу соответствия начала и конца списка. В контексте русского языка: 1-я буква «а» заменяется на 33-ю букву «я», 2-я буква «б» — на 32-ю букву «ю» и так далее.
- Этапы реализации:
 - Формирование последовательности прямого алфавита.
 - Генерация последовательности инвертированного алфавита.
 - Обработка входного текста: определение позиции символа в прямом алфавите и извлечение соответствующего символа из зеркального по тому же индексу.

- Данный алгоритм обладает свойством симметричности: процессы шифрования и дешифрования идентичны.

2.5 шифра Атбаш компилятора

```
[6]
✓ 0
def atbash_cipher(text, alphabet):
    """
    此函数实现了阿特巴什密码，其核心在于利用一个完全反转的字母表来进行镜像替换。
    Эта функция реализует шифр Атбаш, суть которого заключ
    """
    result = ""
    # 我们首先通过 Python 的切片操作快速创建一个与原始顺序完全相反的镜像字母表。
    # Сначала мы создаем зеркальный алфавит с помощью опер
    reversed_alphabet = alphabet[::-1]

    for char in text:
        if char.lower() in alphabet:
            # 第一步，我们找到原字符在标准顺序字母表中的位置索引。
            # Шаг 1: мы находим индекс исходного символа в
            idx = alphabet.find(char.lower())

            # 第二步，我们直接从镜像字母表中取出相同位置的字符，从而完成“首尾对应”的加密。
            # Шаг 2: мы берем символ из зеркального алфавит
            new_char = reversed_alphabet[idx]
            result += new_char.upper() if char.isupper() else new_char
        else:
            # 对于不在字母表内的符号（如数字或空格），程序会跳过加密逻辑并保持其原始状态。
            # Для символов вне алфавита (цифры или пробелы)
            result += char

    return result
```

Рис. 2.3: шифра Атбаш компилятора

2.6 шифра Атбаш результат

```
# --- 演示与测试逻辑 / Демонстрация и тестирование ---

# 我们定义一个完整的俄语字母表，包含33个字母，作为程序运行的基础词库。
# Мы определяем полный русский алфавит из 33 букв,
ru_alphabet = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"

# 这里演示凯撒密码：输入明文和密钥，函数将按照我们解释的位移逻辑生成密文。
# Здесь демонстрация шифра Цезаря: вводим текст и
print("Caesar:", caesar_cipher("Пришел увидел победил", 3, ru_alphabet))

# 这里演示阿特巴什密码：只需输入明文，函数将通过镜像映射生成加密后的结果。
# Здесь демонстрация шифра Атбаш: достаточно ввес
print("Atbash:", atbash_cipher("Саратов", ru_alphabet))

... Caesar: Тулызо целжзо тсцзжло
Atbash: Няоямрз
```

Рис. 2.4: шифра Атбаш результат

3 Вывод

В результате выполнения лабораторной работы были успешно реализованы шифр Цезаря с поддержкой произвольного ключа и шифр Атбаш, основанный на зеркальном отображении. Эксперимент подтвердил, что, несмотря на простоту реализации и прозрачность логики, шифры простой замены обладают низкой криптостойкостью и уязвимы для частотного анализа. В современной криптографии данные алгоритмы используются преимущественно в качестве базовых моделей для изучения принципов кодирования.